AI BLOGNEST API

PROJECT TITLE: BlogNest API - AI-POWERED CONTENT MANAGEMENT AND AUTOMATED BLOG GENERATION SYSTEM PROJECT

- **TEAM ID : SWTID-2026-4496**
- **TEAM LEADER:**

  Sreesha .M.P – (mpsreesha25@gmail.com)

- **TEAM MEMBERS :**

  i. Naga Navaneetha.G - (naganavaneethag@gmail.com)
  ii. Lakshmi Priya .A - (lakshmipriya11012007@gmail.com)
  iii. Divya Dharshini.S –(divya9884297232@gmail.com)
  iv. Leena .M – (leenu2057@gmail.com)

DEPARTMENT: Department of Computer Science

INSTITUTION NAME: S. A. College of Arts and Science

ACADEMIC YEAR: 2026 – 2027

1. EXECUTIVE SUMMARY & INTRODUCTION

The ContentGen backend is a powerful, secure, and intelligent REST API platform designed to simplify blog creation, content management, and AI-assisted writing. Built with Node.js and Express.js, the application uses MongoDB as its primary database with Mongoose ODM for efficient data modeling, validation, and seamless database interactions.

The platform integrates Google Gemini AI to provide intelligent features such as AI-powered blog content generation, AI FAQ assistance, AI WeatherWise insights, and AI FitTrack recommendations, enabling users to create high-quality, context-aware content with minimal effort.

Security is a core aspect of the architecture, implementing JWT (JSON Web Tokens) for stateless authentication, role-based access control (Admin, Editor, Author, Reader) for authorization, and bcrypt.js for secure password hashing. The backend also supports comprehensive blog management, categories and tags, comment management, media metadata handling, advanced search capabilities, and centralized error handling. Additional security mechanisms such as request validation, input sanitization, rate limiting, and structured logging ensure the platform remains reliable, scalable, and resilient against common web vulnerabilities.

Designed with a modular and scalable architecture, ContentGen serves as a modern AI-powered Content Management System (CMS) that combines traditional blogging capabilities with Generative AI to deliver a smarter, faster, and more personalized content creation experience.

2. PROBLEM STATEMENT & SCENARIO-BASED CASE STUDY

Background

Priya is a content creator who manages multiple blogs but struggles to organize posts, schedule publications, manage categories/tags, and track reader engagement efficiently

problems Faced

- Difficulty managing multiple drafts, categories, and tags
- Limited visibility into post performance and engagement analytics .
- Time-consuming processes for content approval and comment moderation
- No streamlined system for handling media uploads or metadata
- Lack of collaborative permission tiers for editors, authors, and administrators

Solution

The BlogNest backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems [cite: 1]. It introduces an internal permission matrix controlling specialized services for content scheduling, category taxonomy, media path registration, and granular user roles

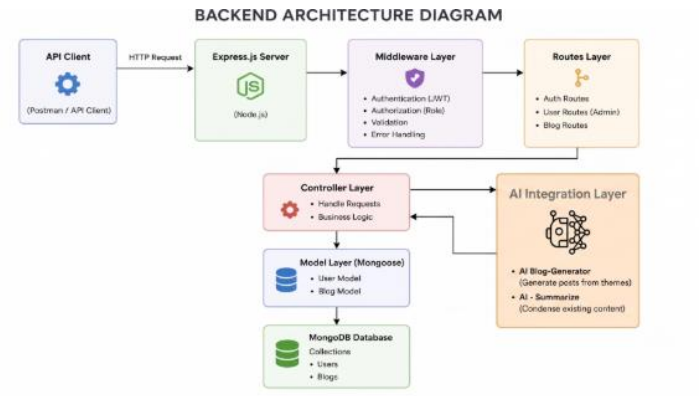
3. SYSTEM REQUIREMENTS & HARDWARE/SOFTWARE SPECIFICATIONS

- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations)
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure [cite: 1].
- **npm (v8 or above):** Node package manager to control operational dependencies
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics
- **Postman / Thunder Client:** API verification toolkit to assert schema validation responses across administrative route shields
- **Code Editor:** Visual Studio Code or similar IDE environment

Hardware Requirements:

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent [cite: 1].
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes)
- **Storage:** 1 GB of available disk workspace

4. PROJECT ARCHITECTURE & SYSTEM DESIGN

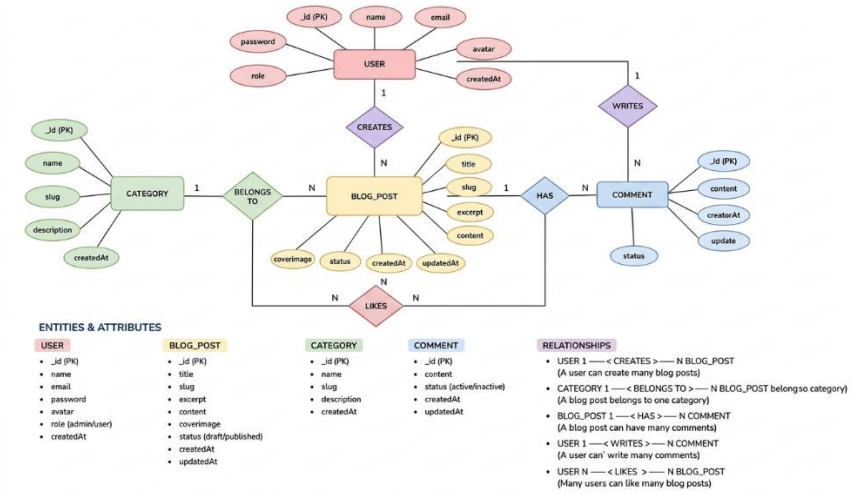


The BlogNest application exposes structured endpoints handled by a layered server architecture model where incoming requests undergo filter processing before interacting with data services

Components Definition:

- **API Client:** Postman tools query distinct route paths to extract JSON entities
- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers
- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries
- **AI Flow (Service Integration):** Connects to external AI models via specialized endpoints (/api/ai/generate-blog and /api/ai/summarize) to automate blog generation and text summarization

5. ENTITY-RELATIONSHIP (ER) MODEL & DATABASE SCHEMA



The underlying document structure manages several collections bound by logical application references

1. User Entity

- id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Post Entity

- userID (Foreign Key referencing User Collection)
- name (String, Required - represents Title)
- photo (Object/String pathway link)
- message (String - Core content body)
- likes (Number, Default: 0)
- timestamps (True)

Key Structural Mappings:

- **User to Post (One-to-Many):** An author can submit multiple blog documents
- **User to Comment (One-to-Many):** Readers/Users can append multiple response comments across independent post nodes

- **Post to Comment (One-to-Many):** A single blog document stores individual referenced comment submissions

6. CORE FEATURES & CAPABILITIES

- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam) .
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections .
- **AI-Driven Blog Generation Pipeline:** Integrates external LLM abstractions to synthesize full-length, structured blog posts from high-level thematic inputs
- **Intelligent Content Summarization Engine:** Leverages semantic compression layers to parse extensive article payloads and dynamically generate concise abstracts

7. ROLES AND RESPONSIBILITIES MATRIX

- **Admin:** Complete environment override control. Can manage user database registrations, alter role assignments, prune malicious comments, and inspect full logging entries
- **Editor:** Focuses on editorial workflows. Reviews pending draft posts, marks records for instant publication, and handles category cleanup scripts .
- **Author:** Restricted account access. Can issue creation statements only for posts bound to their explicit account record
- **Reader:** Consumer access profile. Reads published materials and posts comments under an ongoing authenticated identity

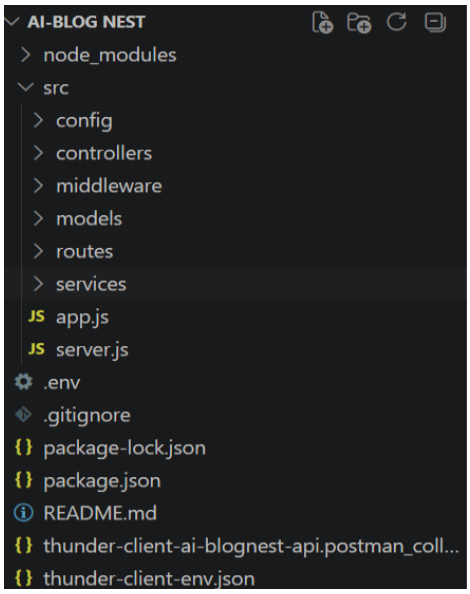
8. MVC ARCHITECTURE PATTERN

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components :

- **Model Layer (Data Blueprint):** Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB
- **Controller Layer (Execution Matrix):** The intermediary brain capturing request vectors from execution routes, checking structural constraints, and packaging results into standard JSON
- **View Layer (Routing Network Interface):** In this headless backend service context, the interface layer lives as an API Routing Layer linking network requests to controller handlers .

9. PROJECT SETUP & CONFIGURATION

1. Create a new folder with your project name.
2. Inside that folder, create a folder named Server
3. Open the folder in VS Code and initialize the project
4. Create necessary directories: src/config, src/controllers, src/middleware, src/models, src/routes, src/services



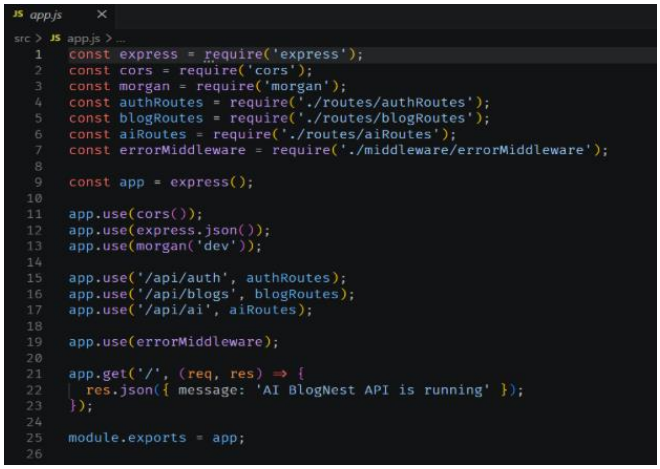
10. BACKEND IMPLEMENTATION

- **Server Setup (server.js):**



```
server.js
src > server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10   console.log(`Server running on port ${PORT}`);
11 });
12
```

- **App Setup (app.js):**



```
app.js
src > app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
26
```

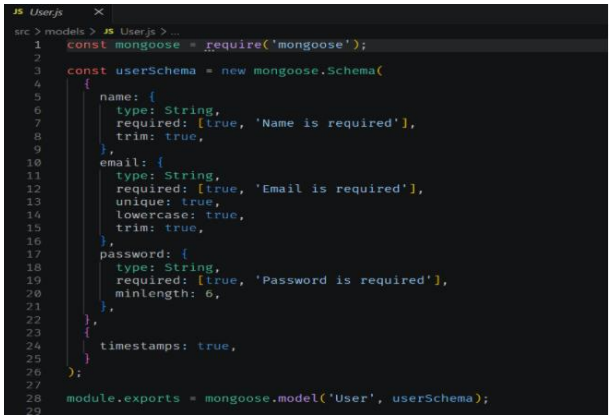
11. DATABASE CONFIGURATION & MONGOOSE SCHEMAS

- **Database Connection (config/db.js):** Establishes a secure connection to MongoDB using Mongoose with proper error handling and environment variable checks



```
Explorer (Ctrl+Shift+E)
src > config > db.js > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
```

- **User Schema (models/User.js):** Enforces field criteria validations for user registration (name, email, unique constraints, hashed passwords, and roles)



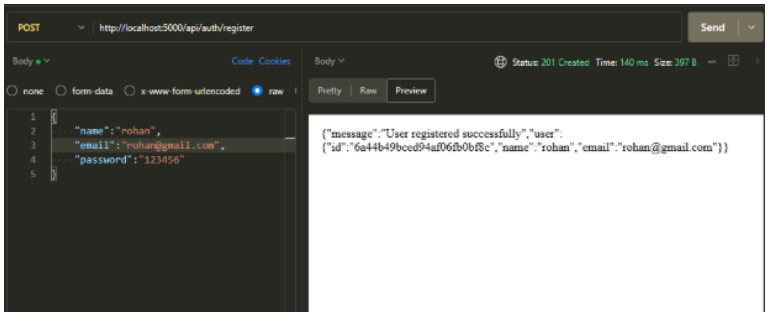
```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: 6,
    },
  },
  {
    timestamps: true,
  }
);

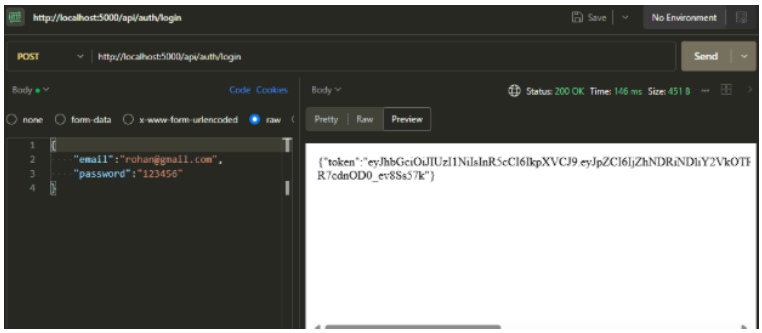
module.exports = mongoose.model('User', userSchema);
```

12. API ENDPOINTS & TESTING

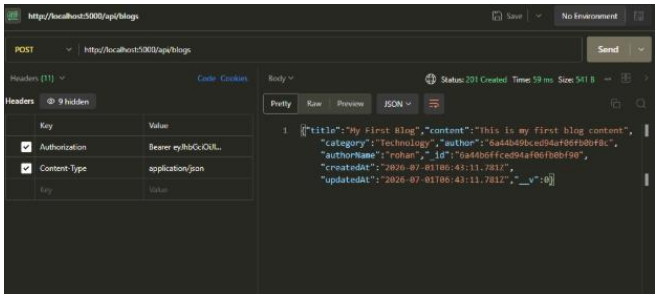
- **User Registration:**



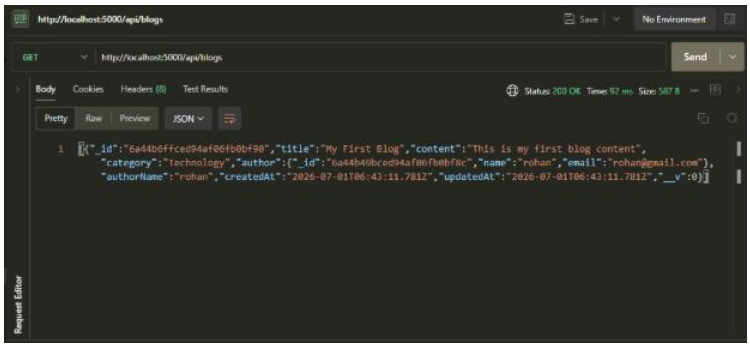
- **User Login:**



- **Create Blogs:**

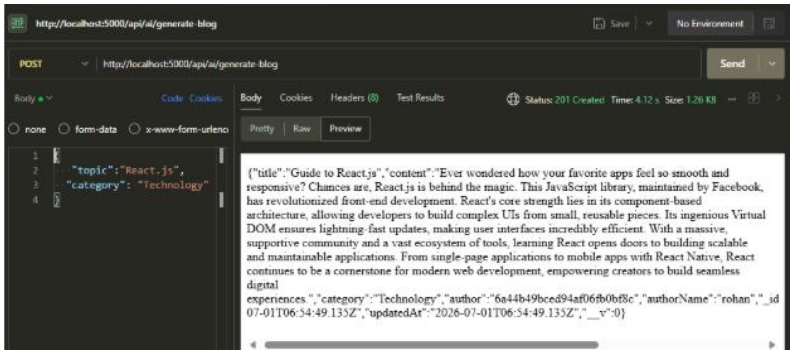


- **Fetch Blogs:**

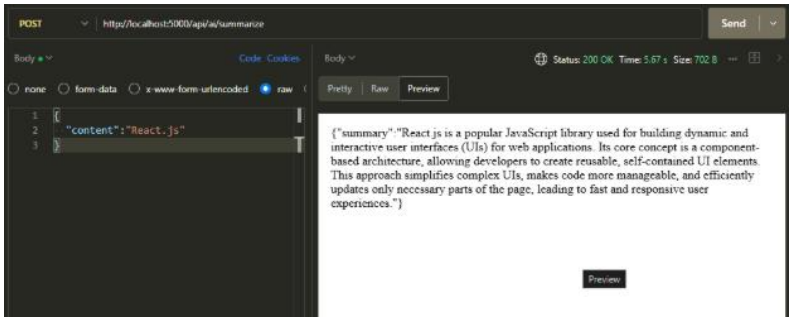


13. AI INTEGRATION LAYER & SERVICES

- **Generate AI Blogs:**



- **Summarize Blogs:**



14. CONCLUSION & FUTURE ENHANCEMENTS

The **ContentGen (BlogNest)** API successfully bridges traditional CMS capabilities with cutting-edge Generative AI features [cite: 1]. Future iterations can incorporate advanced analytics dashboards, real-time WebSocket comment notifications, and enhanced multi-model AI routing to further scale enterprise content creation workflows.

15. DEMO LINK

Demo Video: https://drive.google.com/file/d/1m1u5xUZ0iSnNoeufw4lk5xu1l-hGpBS3/view?usp=drivesdk